

```
from google.colab import files
uploaded = files.upload()
```

telco.csv

telco.csv(text/csv) - 1932822 bytes, last modified: 7/26/2026 - 100% done
Saving telco.csv to telco.csv

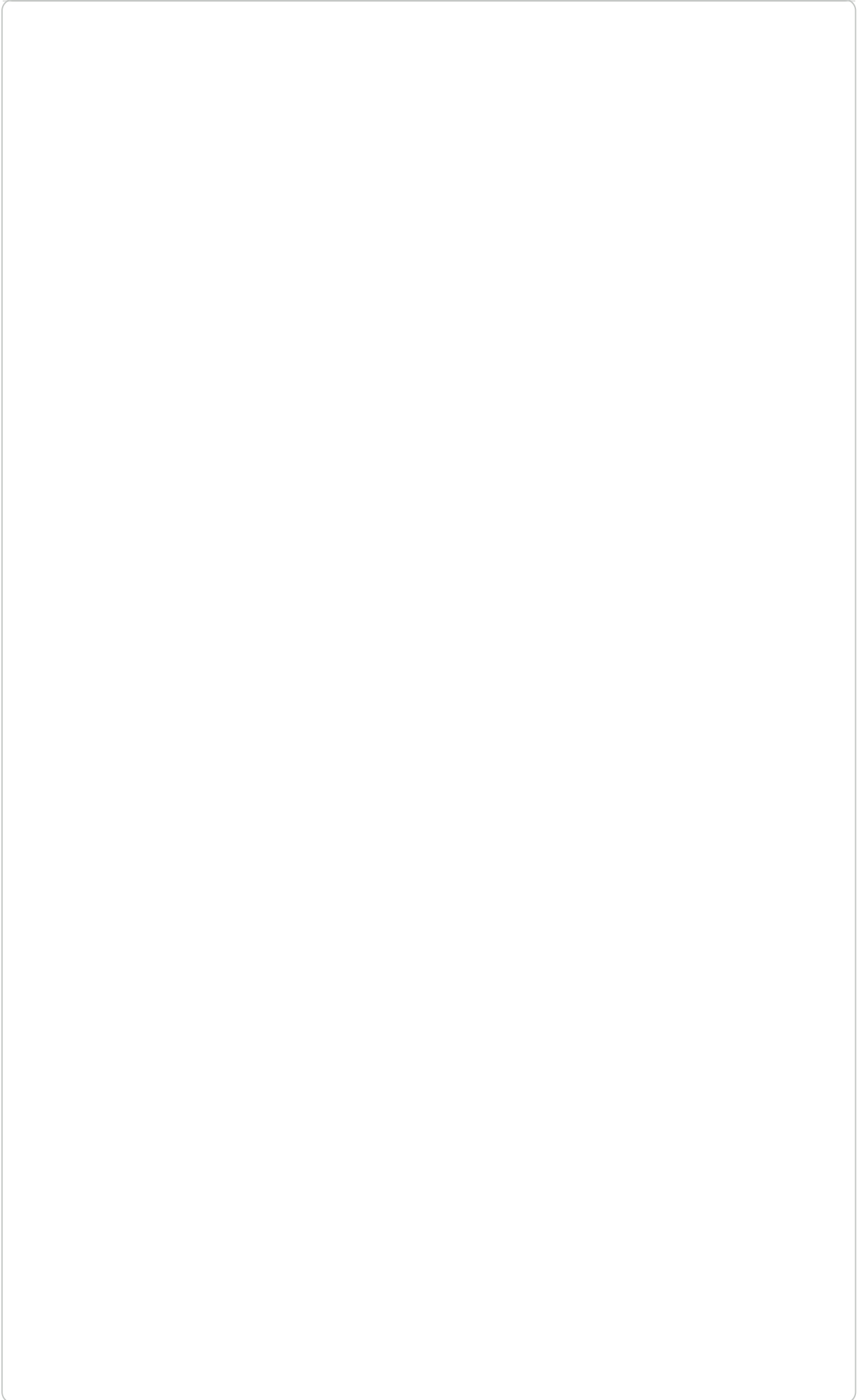
```
import pandas as pd
```

```
df = pd.read_csv('telco.csv')
df.head()
```

	Customer ID	Gender	Age	Under 30	Senior Citizen	Married	Dependents	Number of Dependents	Country
0	8779-QRDMV	Male	78	No	Yes	No	No	0	USA
1	7495-LOOKFY	Female	74	No	Yes	Yes	Yes	1	USA
2	1658-BYGOY	Male	71	No	Yes	No	Yes	3	USA
3	4598-XLKNJ	Female	78	No	Yes	Yes	Yes	1	USA
4	4846-WHAFZ	Female	80	No	Yes	Yes	Yes	1	USA

5 rows × 50 columns

```
df.info()
df.describe()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 50 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Customer ID                               7043 non-null   object
1   Gender                                    7043 non-null   object
2   Age                                        7043 non-null   int64
3   Under 30                                  7043 non-null   object
4   Senior Citizen                            7043 non-null   object
5   Married                                   7043 non-null   object
6   Dependents                               7043 non-null   object
7   Number of Dependents                     7043 non-null   int64
8   Country                                  7043 non-null   object
9   State                                    7043 non-null   object
10  City                                      7043 non-null   object
11  Zip Code                                 7043 non-null   int64
12  Latitude                                7043 non-null   float64
13  Longitude                               7043 non-null   float64
```

```
# Remove unnecessary columns (if exists)
if 'customerID' in df.columns:
    df.drop('customerID', axis=1, inplace=True)

# Convert Total Charges to numeric
df['Total Charges'] = pd.to_numeric(df['Total Charges'], errors='coerce')

# Fill missing values
df.fillna(df.median(numeric_only=True), inplace=True)
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
```

```
33  Unlimited Data                               7043 non-null   object
```

```
import seaborn as sns
import matplotlib.pyplot as plt

# Churn count
sns.countplot(x='Churn Label', data=df)
plt.show()
```

```
# Correlation heatmap
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.show()
```

```
47  CLTV                               7043 non-null   int64
48  Churn Category                     1869 non-null   object
49  Churn Reason                       1869 non-null   object
dtypes: float64(8), int64(11), object(31)
memory usage: 2.7+ MB
```

Age Number of Zip Code Latitude Longitude Pa

Dependents

count	7043.000000	7043.000000	7043.000000	7043.000000	7043.000000	7043.000000
mean	46.509726	0.468692	93486.070567	36.197455	-119.756684	2211.000000
std	16.750352	0.962802	1856.767505	2.468929	2.154425	2111.000000
min	19.000000	0.000000	90001.000000	32.555828	-124.301372	2311.000000
25%	32.000000	0.000000	92101.000000	33.990646	-121.788090	2311.000000
50%	46.000000	0.000000	93518.000000	36.205465	-119.595293	1751.000000
75%	60.000000	0.000000	95329.000000	38.161321	-117.969795	3611.000000
max	80.000000	9.000000	96150.000000	41.962127	-114.192901	10521.000000

5000

```
from sklearn.model_selection import train_test_split
```

```
X = df.drop('Churn Label', axis=1)
```

```
y = df['Churn Label']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
LogisticRegression
LogisticRegression(max_iter=1000)
```

City
Latitude
Population

1.00
0.75
0.50

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
print("Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9758694109297374
```

```
Confusion Matrix:
```

```
[[995 14]
 [ 20 380]]
```

```
Report:
```

	0	1	accuracy	macro avg	weighted avg
0	0.98	0.95	0.97	0.97	0.98
1	0.96	0.95	0.97	0.97	0.98

```
df['Risk'] = model.predict(X)
```

```
# 0 = Loyal, 1 = At Risk
```